

카테고리 중복 오류 설명

09/06/2026 14:19:19

<https://chatgpt.com/g/g-p-69a8fc0a720881918ee40d1fc7e9bda5-trend-leader-kaebseuton/c/6a990eb7-7714-83ee-8bfc-795a1f15f212>

Thinking ● 정리하기 현재 대화에서는 1번과 2번을 임시 결정으로 반영하고, 3번은 ...



Thinking

- 정리하기

현재 대화에서는 1번과 2번을 임시 결정으로 반영하고, 3번은 보류하는 흐름이 적절해 보여요. 저는 다음 단계로 이 결정들을 문서화한 뒤, 트렌드 속성 설계를 별도로 구체화하겠습니다.

- 정리했다

✓ 14s 동안 처리함

좋습니다. 그러면 현재 채팅에서 합의된 내용은 설계 기준으로 유지하되, **Trend Attribute**만 미확정 상태로 남기는 것이 가장 깔끔합니다.

현재까지는 이렇게 정리할 수 있습니다.

[확정 방향]

1. Category는 "무슨 분야인가?"만 표현한다.
 - 대분류: 게임, 음식, 패션 ...
 - 세부분류: 모바일 게임, 한식, 스킨케어 ...
2. 사용자는 대분류만 관심사로 선택한다.
3. Trend에는 가능한 가장 구체적인 Category만 매핑한다.
 - 모바일 게임 Trend라면 GAME + 모바일 게임을 중복 저장하지 않음
 - 모바일 게임만 저장
4. 맞춤 추천에서는
사용자 대분류 → 하위 Category 범위
를 따라 Trend를 찾는다.
5. category_name의 전역 UNIQUE는 제거하는 방향으로 간다.
 - 다른 부모 아래 같은 이름 허용
 - 같은 부모 아래 같은 이름은 금지
 - 단, 이 규칙의 실제 MariaDB Constraint 표현 방식은 별도 확정 필요
6. 신제품 / 업데이트 / 이벤트 / 밈 등은 Category에 넣지 않는다.
7. Hashtag도 Category에 넣지 않는다.

8. AI가 생성·추천한 Hashtag는

`trend_related_keywords + HASHTAG`
구조를 유지한다.

9. 실제 외부 Source에서 관측된 Hashtag는

AI Analysis에 종속시키지 않는다.
- 실제 수집 요구가 생기면 source 기반 별도 구조를 설계한다.

현재 Frozen Schema가 카테고리 계층을 `parent_id`로 표현하고, 사용자의 관심사는 대분류만 허용하고 있기 때문에 1~4번은 기존 설계를 파괴하기보다 그동안 암묵적이었던 의미를 더 명확하게 만드는 방향입니다. `schema_decisions.md` `Trend_Leader_관심사_조회_및_수정_설계_확정안_v1.0.2.md`

다만 5번은 기존 `categories.category_name UNIQUE`라는 Frozen 결정 자체를 변경하는 것이므로, 나중에 실제 SQL/Alembic 단계에 들어갈 때는 정식 Schema Decision으로 남기는 게 맞습니다.

`schema_decisions.md`

이제 3번, Trend Attribute를 어떻게 볼 것인가

여기서부터는 바로 컬럼을 만들면 안 될 것 같습니다. 먼저 **Attribute**가 대체 무엇을 답하려는 데이터 인지를 정해야 합니다.

예를 들어 다음 후보를 보면:

신제품
업데이트
이벤트
ميم
논란
출시
콜라보
대회

겉보기에는 모두 "트렌드 종류"처럼 보이지만 사실 서로 다른 축이 섞여 있습니다.

신제품 / 출시
→ 상품·출시 상태

업데이트
→ 기존 제품의 변경 이벤트

이벤트 / 대회
→ 행사 유형

میم

→ 확산 형태

논란

→ 사회적 반응/이슈 상태

콜라보

→ 관계/마케팅 형태

그래서 단순히:

```
trend_type = ENUM(...)
```

으로 넣으면 시간이 지나면서 굉장히 큰 잡탕 Enum이 될 가능성이 있습니다.

생각해볼 수 있는 설계는 크게 3가지입니다

A. Trend 하나당 단일 Type

예:

```
trend_type = UPDATE
```

가장 단순합니다.

하지만:

"원신 신규 캐릭터 업데이트 이벤트"

같은 Trend는

UPDATE?

EVENT?

둘 중 하나를 억지로 골라야 합니다.

MVP 구현은 쉽지만 의미 손실이 큼니다.

B. Trend에 여러 Attribute를 붙이는 구조

개념적으로:

```
Trend
├─ UPDATE
├─ EVENT
└─ COLLABORATION
```

처럼 M:N으로 만듭니다.

예를 들면 장기적으로:

```
trend_attributes
```

```
attribute_id
attribute_code
attribute_name
```

```
trend_attribute_map
```

```
trend_id
attribute_id
```

가 될 수 있습니다.

확장성은 가장 좋습니다.

다만 지금 단계에서 실제 요구가 많지 않다면 **카테고리에 이어 또 하나의 Taxonomy 시스템을 만드는 것**이라 과설계가 될 수도 있습니다.

C. MVP에서는 **Attribute** 자체를 **DB**에 만들지 않는다

현재 이미 존재하는:

```
AI related keywords
AI hashtags
Trend title / summary
```

만으로 우선 구현하고,

실제로 다음 요구가 생길 때 **Attribute**를 추가합니다.

"업데이트 Trend만 보고 싶다"
"신제품 Trend를 필터링한다"
"میم만 모아 보여준다"
"유형별 통계를 낸다"

즉 **필터링·추천·통계**에 실제로 쓰일 때 **구조화 데이터**로 승격시키는 방식입니다.

현재 MVP 관점에서는 이것도 상당히 합리적입니다.

지금 제가 가장 경계하는 구조

이런 식입니다.

```
Category
게임
└─ 모바일게임
└─ 업데이트
└─ 신제품
└─ 밈
```

이건 이제 확실히 피하기로 했습니다.

그런데 다음 단계에서 실수하기 쉬운 것이:

```
trend_type =
신제품 | 업데이트 | 이벤트 | 밈 | 논란 | 콜라보 | ...
```

를 아무 기준 없이 하나의 Enum에 전부 넣는 것입니다.

그렇게 되면 Category에서 피했던 **서로 다른 분류 축 혼합**을 Trend Attribute에서 다시 반복하게 됩니다.

그래서 다음 설계에서는 먼저 실제 사례를 가지고 분류해보는 게 좋겠습니다

예를 들어 Trend Leader에서 충분히 들어올 법한 Trend를 몇 개 놓아보면:

- ① 아이폰 신제품 공개
- ② 리그 오브 레전드 신규 패치
- ③ 두바이 초콜릿 품질 대란
- ④ 유명 연예인 패션 밈 확산

- ⑤ 게임 × 애니메이션 콜라보 이벤트
- ⑥ 신규 모바일 RPG 출시
- ⑦ 화장품 브랜드 신제품 논란

여기에 우리가 만든 Attribute 후보를 붙여보는 겁니다.

예:

- ① 신제품 / 발표
- ② 업데이트
- ③ 유행? 품질? 바이럴?
- ④ 밈
- ⑤ 콜라보 / 이벤트
- ⑥ 출시 / 신작
- ⑦ 신제품 / 논란

벌써 한 Trend가 여러 Attribute를 갖는 사례가 자연스럽게 발생합니다.

이 테스트 결과만 보면 **단일 ENUM 방식은 조금 위험하다**는 힌트가 있습니다.

하지만 아직 M:N 테이블이 반드시 필요하다고 확정할 단계도 아닙니다.

다음 단계에서 같이 결정할 질문은 두 개면 충분합니다

첫 번째는:

Trend Attribute를 사용자가 실제로 필터링하거나 추천 기준으로 사용할 것인가, 아니면 화면에서 설명용 메타데이터로만 사용할 것인가?

이 답에 따라 DB에 정규화해서 저장할 필요성이 크게 달라집니다.

두 번째는:

"업데이트 + 이벤트", "신제품 + 논란" 처럼 하나의 Trend에 여러 Attribute가 동시에 존재하는 것을 정상 상태로 인정할 것인가?

여기서 YES 라면 단일 trend_type Enum보다는 다른 구조가 훨씬 자연스러워집니다.

현재 단계에서는 **Category/Uniqueness/Hashtag** 방향은 유지하고, **Attribute는 Schema에 아직 넣지 않는 것**을 권합니다. 먼저 이 두 질문을 가지고 사례 기반으로 설계한 다음, 그 결과까지 모아서 새로운 Schema Decision을 작성하는 순서가 가장 안전합니다.

2026. 9. 6. 오후 2:16:41